

Attention-Based Seq2Seq: End-to-End Workflow

Perfect lets now build a fresh, comprehensive, and fully correct attention-based sequence-to-sequence (Seq2Seq) workflow for translation, from embedding to attention-based decoding, using:

- Clear vectorized math steps
- A final ASCII dashed-line diagram that ties everything together

Hypothetical Example

Input (English): "The cat eats"

Output (French): "Le chat mange"

Vocabulary and Embedding Setup

Input (English) tokens:

Word	Index	Embedding
The	1	[0.1, 0.3]
Cat	2	[0.2, 0.4]
Eats	3	[0.6, 0.9]

$X = [x_1, x_2, x_3] = [[0.1, 0.3], [0.2, 0.4], [0.6, 0.9]]$ # shape: (3, 2)

Output (French) tokens:

Word	Index	Embedding
<START>	0	[0.0, 0.0]
Le	1	[0.3, 0.4]
Chat	2	[0.2, 0.5]
Mange	3	[0.6, 0.1]

Step-by-Step Vectorized Implementation

1. Encoder (Bi)LSTM forward only here for clarity

Assume:

- Hidden size: 1 (scalar)
- Weights: $W_{x_enc} = [0.5, 0.1]$, $W_{h_enc} = 0.3$

Forward Pass:

$h_1 = \tanh(0.5 \cdot 0.1 + 0.1 \cdot 0.3 + 0) = \tanh(0.08) = 0.08$

$h_2 = \tanh(0.5 \cdot 0.2 + 0.1 \cdot 0.4 + 0.3 \cdot 0.08) = \tanh(0.164) = 0.162$

$h_3 = \tanh(0.5 \cdot 0.6 + 0.1 \cdot 0.9 + 0.3 \cdot 0.162) = \tanh(0.4386) = 0.412$

$H = [h_1, h_2, h_3] = [0.08, 0.162, 0.412]$

2. Decoder Initialization

- Initial state: $s_0 = h_3 = 0.412$
- Decoder input at $t=1$: <START> [0.0, 0.0]
- Decoder weights: $W_{x_dec} = [0.6, 0.2]$, $W_{h_dec} = 0.3$

3. Decoder Step at $t = 1$

Attention-Based Seq2Seq: End-to-End Workflow

(a) Compute decoder hidden state s_1 :

```
s1 = tanh(Wxy0 + Whs0)
= tanh(0 + 0.30.412) = tanh(0.1236)  0.123
```

(b) Compute alignment scores:

```
e1j = s1  hj = [0.0098, 0.0199, 0.0507]
```

(c) Softmax attention weights:

```
1j = softmax(e1j) = [0.327, 0.331, 0.341]
```

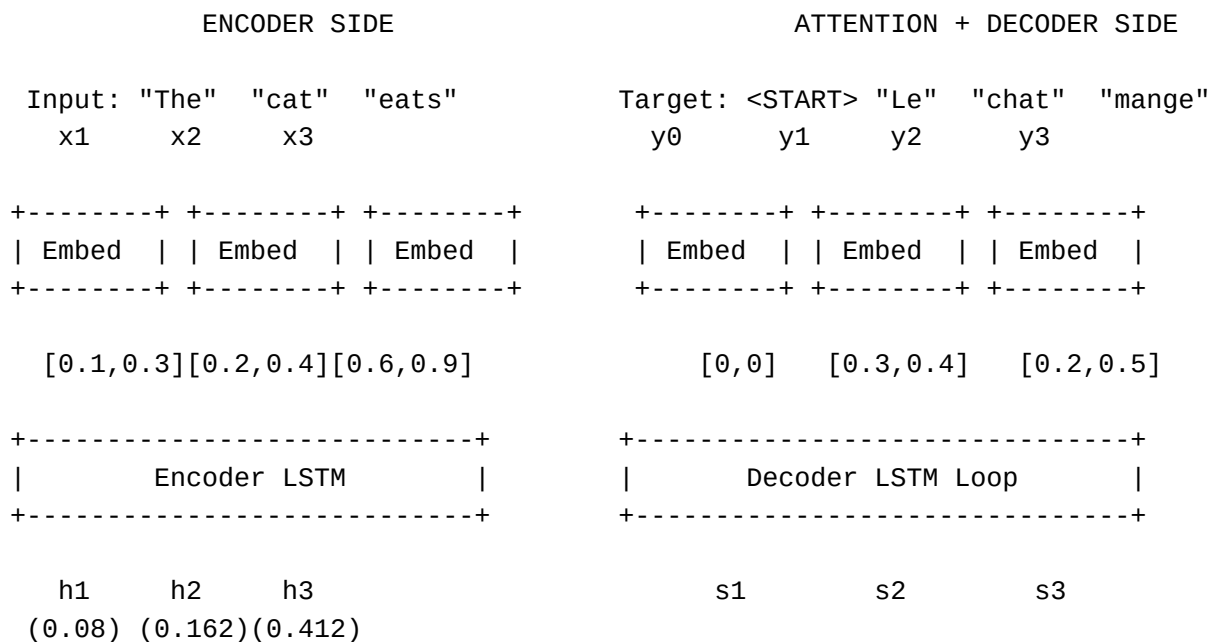
(d) Compute context vector:

```
c1 = 1jhj = 0.2204
```

(e) Combine & Predict output:

```
z = 0.7c1 + 0.3s1 = 0.1909
W_vocab = [0.1, 0.3, 0.6]
scores = z  W_vocab = [0.0191, 0.0573, 0.1145]  softmax  chat
```

ASCII Diagram of Full Architecture



Attention Mechanism

$e_{tj} = s_t \cdot h_j$ $t_j = \text{softmax}$

t_j applied to h_j

```
+-----+
| Context vector ct|  weighted sum of encoder outputs
+-----+
```

```
[st  ct]  Dense + softmax  yt
```

Output word: Le, chat, mange ...